

CHECK WHAT YOU LEARNT

Check Yourself - Unit 3**A. Choose the correct answer**

- The middle leg of a potentiometer is called the
(a) anode (b) wiper (c) trimmer (d) ground
- A pot with the knob halfway gives an analogRead of about
(a) 0 (b) 255 (c) 512 (d) 1023
- To fade an LED you must use
(a) digitalWrite (b) analogWrite (c) digitalRead (d) delay
- In an if - else if - else ladder, the conditions are checked
(a) all at once (b) in order, top to bottom (c) backwards (d) at random
- The pot's two outer legs connect to
(a) A0 and D9 (b) 5V and GND (c) GND and A0 (d) two LEDs
- Reading jumps straight from 0 to 1023. The likely fault is
(a) a burnt LED (b) the wiper on an outer leg (c) wrong baud rate (d) a missing resistor

B. Fill in the blanks

- A potentiometer works as a _____ divider.
- map(raw, 0, 1023, 0, 255) converts the range so that _____ can use it.
- Three equal zones across 0 to 1023 split at about _____ and _____.
- LEDs used for fading must be on pins marked with _____.

C. Answer in one or two lines

- Explain how turning a knob changes the voltage on the wiper.
- Why does the second rung of the ladder not need to say 'greater than 341'?
- Write down the raw values at which your own LEDs changed over.
- Name two machines in daily life that use a potentiometer.

UNIT 4**Intermediate Project: Temperature and Humidity Monitor**

Sessions 9, 10 and 11

Now a real sensor replaces your hand on the knob. The DHT11 measures two completely different quantities from one small blue box, and your machine has to decide what those numbers mean for the people sitting in the room.

By the end of this unit you will be able to:

- Install and use an external Arduino library.
- Read two different values from a single sensor.
- Build a three-band decision ladder from real measured data.
- Explain why a small gap between switching points prevents flickering.

SESSION 9

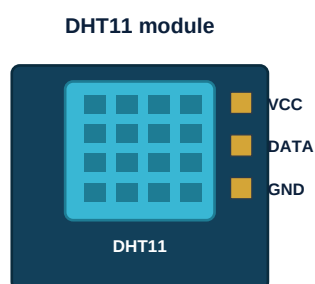
one class

The DHT11 and Your First Library

Today you will

- Learn what the DHT11 measures and how accurate it is.
- Install a library from the Library Manager.
- Print live temperature and humidity to the Serial Monitor.

Inside the blue plastic grille of a DHT11 there are two tiny sensors. One measures temperature. The other measures humidity - the amount of water vapour held in the air, given as a percentage. A small chip beside them packages both readings into a coded pulse and sends it out of a single DATA pin.



One sensor, two numbers

Temperature

how hot the air is, in degrees Celsius
useful range: 0 to 50 C

Humidity

how much water vapour the air holds, as a percentage
useful range: 20 to 90 %

The DHT11 is a digital sensor, so it goes to a digital pin - not an analog one. A library does the decoding for you.

Figure 4.1 - The DHT11 measures two different things at once

That coded pulse is far too complicated to decode by hand, and this is where a **library** comes in. A library is a bundle of instructions somebody else has already written and tested, which you are free to use. Instead of a hundred lines of timing code, you write `dht.readTemperature()` and you get a number.

- 1 In the IDE, open **Tools > Manage Libraries**.
- 2 Type **DHT sensor library** in the search box.
- 3 Choose the one by **Adafruit** and press Install.
- 4 If it offers to install **Adafruit Unified Sensor** as well, say yes. It needs it.
- 5 Close the manager. The new instructions are now available to every sketch.

ReadDHT11.ino

```
#include <DHT.h>           // bring in the library

#define DHTPIN 4           // DATA pin of the sensor
#define DHTTYPE DHT11      // which family of sensor it is
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float t = dht.readTemperature(); // degrees Celsius
  float h = dht.readHumidity();    // percentage
```

```

Serial.print("Temperature: ");
Serial.print(t);
Serial.print(" C   Humidity: ");
Serial.print(h);
Serial.println(" %");

delay(2000);          // the DHT11 needs 2 seconds between reads
}

```

Watch Out

The delay of 2000 is not optional. A DHT11 physically cannot produce a fresh reading faster than once every two seconds. Ask it more often and it hands you the old value, or the word **nan**, which stands for *not a number*.

SESSION 10

one class

Building the Monitor

Today you will

- Wire the DHT11, three LEDs and the buzzer.
- Record real classroom readings across the session.
- Decide your own cold, normal and hot boundaries.

Temperature and humidity monitor - connections

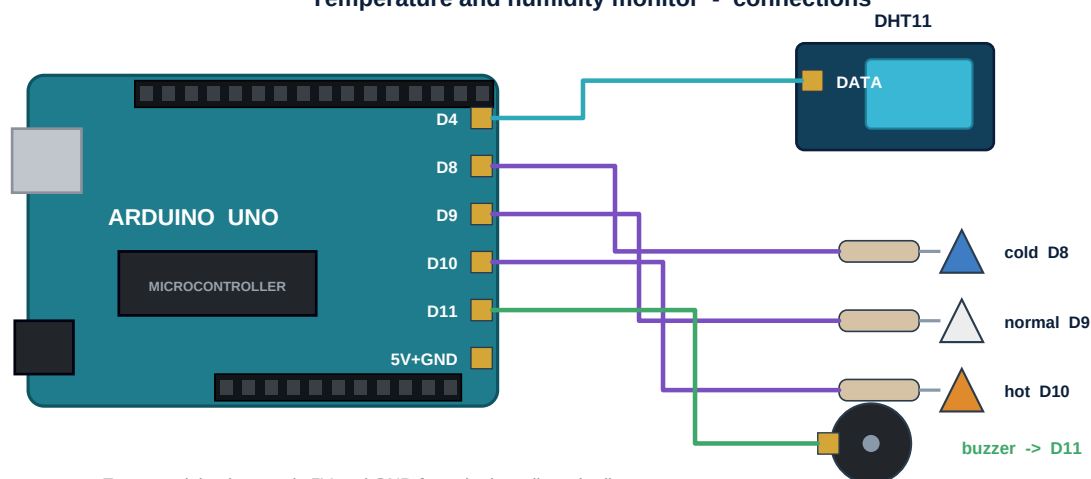


Figure 4.2 - One sensor decides which of three lamps is lit

Component pin	Arduino pin	Why
DHT11 VCC	5V	Power for the sensor
DHT11 GND	GND	Shared return path
DHT11 DATA	D4	The coded reading comes out here
Blue LED long leg	D8 through 220 ohm	Cold indicator
White LED long leg	D9 through 220 ohm	Comfortable indicator
Orange LED long leg	D10 through 220 ohm	Hot indicator
Buzzer +	D11	Alert above the hot limit
Buzzer -	GND	Return path

Before you write a single line of decision code, gather data. A machine designed on guesswork behaves like a machine designed on guesswork. Fill this table during the session.

Where and when	Temperature (C)	Humidity (%)
Classroom, start of the session	_____	_____
Classroom, end of the session	_____	_____
Near an open window	_____	_____
Sensor held in a cupped hand for 30 seconds	_____	_____
Corridor outside	_____	_____

Try This

Breathe gently on the sensor and watch the humidity climb, then fall back over the next minute. Your breath carries a great deal of water vapour. This is also a quick way to prove your sensor is genuinely working and not just repeating a stored number.

SESSION 11

one class

Alert Logic and the Decision Ladder**Today you will**

- Turn your measured data into three temperature bands.
- Add a buzzer alert above the hot limit.
- Stop the LEDs flickering at a boundary.

Now the machine gets an opinion. Using the numbers you collected, pick two boundaries: below the first is cold, above the second is hot, and everything between is comfortable. For most Indian classrooms, 24 and 30 degrees are sensible starting points - but use your own data, not ours.

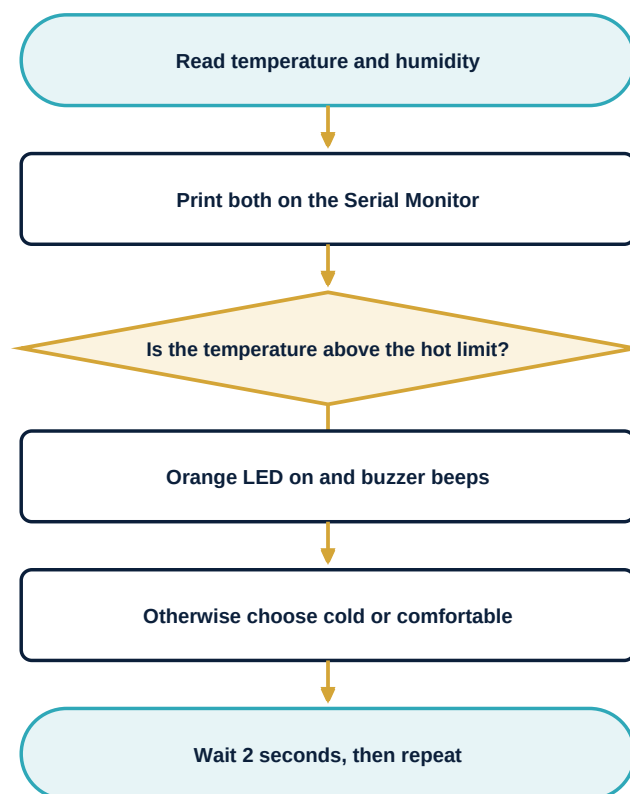


Figure 4.3 - The decision ladder, drawn before it is coded

TempHumidityMonitor.ino

```
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

int coldLed = 8, okLed = 9, hotLed = 10, buzzer = 11;

float coldLimit = 24.0;    // change these after your own testing
float hotLimit = 30.0;

void setup() {
  pinMode(coldLed, OUTPUT);
  pinMode(okLed, OUTPUT);
  pinMode(hotLed, OUTPUT);
  pinMode(buzzer, OUTPUT);
  Serial.begin(9600);
  dht.begin();
}

void allLedsOff() {          // our own instruction
  digitalWrite(coldLed, LOW);
  digitalWrite(okLed, LOW);
  digitalWrite(hotLed, LOW);
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (isnan(t) || isnan(h)) {    // the sensor failed to answer
    Serial.println("Sensor not responding - check the DATA wire");
    delay(2000);
    return;                     // skip the rest of this loop
  }

  Serial.print("Temperature: ");
  Serial.print(t);
  Serial.print(" C   Humidity: ");
  Serial.print(h);
  Serial.println(" %");

  allLedsOff();

  if (t < coldLimit) {
    digitalWrite(coldLed, HIGH);
    digitalWrite(buzzer, LOW);
  } else if (t < hotLimit) {
    digitalWrite(okLed, HIGH);
    digitalWrite(buzzer, LOW);
  } else {
    digitalWrite(hotLed, HIGH);
    digitalWrite(buzzer, HIGH);    // too hot - warn the room
    delay(300);
    digitalWrite(buzzer, LOW);
  }

  delay(2000);
}
```

Two professional touches are hiding in that program. The first is `isnan(t)`, which checks whether the sensor actually answered. Real engineers never assume a sensor replied. The second is `allLedsOff()`, an instruction we wrote ourselves so that the ladder below stays short and readable.

11.1 The Flickering Problem, and Its Real Name

Sit the sensor at exactly 30.0 degrees and watch. The reading wobbles - 29.9, 30.1, 30.0, 30.2 - and the orange LED and the buzzer flicker on and off maddeningly. Real thermostats solve this with a trick called **hysteresis**: the switch-on point and the switch-off point are deliberately made different.

In plain English: once the alarm has started, do not stop it until the temperature has fallen a clear degree below the limit. A one-degree gap is enough to make the machine feel calm and deliberate instead of nervous.

Challenge

Implement hysteresis. Add `boolean alarmOn = false;` at the top. Switch the alarm on when `t > hotLimit`, but switch it off only when `t < hotLimit - 1.0`. Watch the flickering disappear completely.

New word	What it means
Library	Ready-made instructions written by somebody else that you can use.
Humidity	How much water vapour the air is holding, as a percentage.
nan	Not a number - what a sensor returns when it fails to answer.
Hysteresis	Using different switch-on and switch-off points to stop flickering.